

PLAN DE COURS

Automne 2026

INF111 : Programmation orientée-objet (hors programme) (4 crédits)

Préalables

Aucun préalable requis.

Description du cours

Ce cours vise à développer les aptitudes nécessaires pour exploiter judicieusement le paradigme de programmation objet.

Au terme de ce cours, la personne étudiante sera en mesure de :

- acquérir la maîtrise d'un environnement de programmation objet;
- décrire les caractéristiques du langage de programmation;
- appliquer les concepts de classe, d'objet, d'encapsulation, d'héritage et de polymorphisme.
- résoudre des problèmes de programmation en appliquant le paradigme objet;

Éléments de contenu : environnement de programmation. Normes de programmation. Caractéristiques du langage. Classes et objets. Encapsulation. Chaînes de caractères. Tableaux et vecteurs. Types de données abstraits. Chainages simple et double. Collections. Héritage. Polymorphisme. Gestion d'erreurs et d'exceptions.

Stratégies pédagogiques

La première semaine de la session servira à une révision de concepts présumés comme étant déjà connus et à l'apprentissage de ceux-ci en Java. Les cours suivants présenteront les concepts de programmation orientée-objet et des applications de ceux-ci à l'aide de notions de programmation plus approfondies et de l'utilisation des composants *Swing*.

3 heures de cours magistraux par semaine

3 heures de laboratoire par semaine

Les laboratoires visent :

- L'assimilation des notions vues au cours à l'aide d'exercices;
- La mise au point des programmes donnés en travaux pratiques (si les exercices sont finis).

Informations concernant l'agrément du BCAPG

Les objectifs de ce cours sont liés aux indicateurs de qualités requises des diplômés de la manière suivante :

Objectif spécifique	Qualité	Indicateur	Niveau d'enseignement
Acquérir la maîtrise d'un environnement de programmation objet.	.		
Décrire les caractéristiques du langage de programmation.	.		
Appliquer les concepts de classe, d'objet, d'encapsulation, d'héritage et de polymorphisme.	.		
Résoudre des problèmes de programmation en appliquant le paradigme objet.	.		

Utilisation d'appareils électroniques

Pas de calculatrice.

Horaire

Groupe	Jour	Heure	Activité
01	Mercredi Vendredi	09:00 - 12:30 13:30 - 16:30	Activité de cours Laboratoire
02	Mercredi Vendredi	09:00 - 12:00 13:30 - 17:00	Laboratoire Activité de cours
03	Mardi Jeudi	09:00 - 12:30 09:00 - 12:00	Activité de cours Laboratoire
05	Mardi Jeudi	13:30 - 17:00 13:30 - 16:30	Activité de cours Laboratoire

Coordonnées du personnel enseignant le cours

Groupe	Nom	Activité	Courriel	Local	Disponibilité
01	Stéphane Duval	Activité de cours	Stephane.Duval@etsmtl.ca	B-2520	
01	Stéphane Duval	Laboratoire	Stephane.Duval@etsmtl.ca	B-2520	
02		Laboratoire			
02	Asma Bellili	Activité de cours	cc-asma.bellili@etsmtl.ca	B-2568	
03		Laboratoire			
03	Frédéric Simard	Activité de cours	Frederic.Simard@etsmtl.ca	B-2568	
05	Stéphane Duval	Laboratoire	Stephane.Duval@etsmtl.ca	B-2520	
05	Frédéric Simard	Activité de cours	Frederic.Simard@etsmtl.ca	B-2568	

Cours

- La première période de trois heures sert aussi à présenter le plan de cours, l'approche pédagogique utilisée et les différents modèles d'évaluation.
- Notez que l'ordre de présentation peut varier selon l'enseignante ou l'enseignant.

Semaine	MATIÈRE
1	- Présentation du cours : plan de cours, stratégies pédagogiques, évaluations, ... - Environnement de programmation; - Syntaxe Java; - Introduction aux classes (attributs à accès privé) et aux objets; - Survol des notions d'objet : - Classes et objets; - Attributs et méthodes; - Héritage par extension (<i>extends</i>); - Redéfinition (<i>override</i>); - Interface et implémentation (<i>implements</i>).
2 - 4	- Éléments de base de la programmation et syntaxe Java : - Types primitifs (variables et littéraux); - Opérateurs et expressions; - Entrées/sorties (<i>Scanner</i>, <i>System.out</i>); - Structure d'un programme (<i>main()</i>); - Structures de contrôle (conditionnelles et répétitives); - Tableaux 1D et 2D; - Méthodes, transmission de paramètres et retour de résultat (<i>return</i>); - Ruptures de séquence (<i>break</i> et <i>continue</i>) : fonctionnement, risques et règles d'utilisation. Philosophie du programmeur : Le moins coûteux en temps machine, en espace mémoire et en répétition de code sans nuire à sa clarté. - Normes de programmation - Valeurs vs références; - Les paquetages (<i>package</i> et <i>import</i>); - Survol des packages <i>java.lang</i> et <i>java.util</i> de Java; - Classes de Java : <i>String</i>, <i>Arrays</i> et <i>Math</i>;

	• Les chaînes de caractères; • Les API de Java et utilisation de classes existantes; • Tris (à bulles, par sélection, par insertion); • Recherches/fouilles (séquentielle, binaire et séquentielle indexée); • Encapsulation; • Méthodes d'accès (accesseurs et mutateurs) et d'instances; • Tableaux d'objets; • Constructeurs; • Surcharge; • Conversion d'objets en chaînes de caractères; • Comparaison d'objets (test d'égalité - <i>equals()</i>); • Référence <i>this</i>; • Attributs et méthodes de classes (<i>static</i>); • Agrégation/composition; • Documentation du code et commentaires Javadoc.
Examen Intra 1	Contenu des semaines 1 à 4
6 - 8	• Vecteurs (<i>ArrayList</i> et <i>Vector</i>); • Notion de classe générique; • Classes enveloppes (<i>wrappers</i>) et conversion automatique (<i>autoboxing</i> et <i>unboxing</i>); • Itérateurs de listes (<i>ListIterator</i>); • Types de données abstraits (piles, files, listes), implémentation statique. Stratégies de gestion (LIFO et FIFO); • Classes internes (<i>inner class</i>); • Cadre de collection (<i>LinkedList</i>, <i>Stack</i>, <i>Queue</i>,...); • Chainage; • Types de données abstraits (piles, files, listes), implémentation dynamique avec chainages simple et double; • Implémentation d'un itérateur (exemple). • Héritage par extension : ◦ Définition et utilité; ◦ Syntaxe (<i>extends</i>); ◦ Accès protégé (<i>protected</i>); ◦ Appel aux constructeurs et aux méthodes de la classe mère (<i>super</i>); ◦ Hiérarchies d'héritage; ◦ Généralisation et spécialisation; ◦ Héritage vs agrégation/composition (<i>isA</i> vs <i>hasA</i>); ◦ Héritage simple vs héritage multiple; • Polymorphisme : ◦ Définition; ◦ Surcharge (<i>overload</i>); ◦ Redéfinition (<i>override</i>); ◦ Méthodes virtuelles et liaison dynamique (<i>late binding</i>); ◦ Méthodes non polymorphiques; ◦ Méthodes et classes abstraites (<i>abstract</i>); ◦ Méthodes et classes finales (<i>final</i>); • Exemple d'application de l'héritage et du polymorphisme : les graphismes (classe <i>Graphics</i>); • Introduction aux enregistrements (attributs à accès public).
Examen Intra 2	Contenu des semaines 1 à 8
10-13	• Gestion des erreurs et des exceptions; • Clonage et constructeurs de copies; • Interfaces (<i>interface</i>) : ◦ Définition et utilité;

	◦ Implémentation; ◦ Propriétés. • Notion d'interface générique; • Application des interfaces aux tris et aux recherches : ◦ Test d'inégalité, ordre naturel et comparaison d'objets (interface <i>Comparable</i>); ◦ Compérateurs (interface <i>Comparator</i>); ◦ Méthodes de tri et de recherche binaire (méthodes des classes <i>Arrays</i> et <i>Collections</i>); ◦ Notion de programmation par contrats. • Appels entre constructeurs d'une même classe (<i>this()</i>); • Finalisation de la destruction d'objets (<i>finalize()</i>); • Ramasse-miettes; • Types énumérés (<i>enum</i>); • Blocs d'initialisation statique; • Exemple de définition de classes et d'interfaces génériques : implémentation d'un itérateur; • Exemple de mise en oeuvre : la programmation graphique avec <i>Swing</i> ◦ Composants <i>Swing</i> : <i>JFrame</i>, <i>JPanel</i>, <i>JButton</i>, <i>JTextField</i>, <i>JMenu</i>, <i>JList</i>, ...; ◦ Boîtes de dialogue (<i>JOptionPane</i>); ◦ Gestion de disposition (<i>Layout Manager</i>) : <i>BorderLayout</i>, <i>FlowLayout</i>, <i>GridLayout</i>; ◦ Gestion d'évènements : modèle de délégation, interfaces écouteurs (<i>listeners</i>); ◦ Précautions (threads et <i>Swing</i>, <i>SwingUtilities.invokeLater()</i>) ◦ Complément : ■ Introduction à l'architecture MVC (non évalué); ■ Patron observer;
Examen final	Contenu de toute la session.

Laboratoires et travaux pratiques

Laboratoires :

- Chaque semaine des exercices sont proposés pour comprendre la théorie de la semaine en vue de la réalisation des travaux pratiques. Les laboratoires ne sont pas à remettre mais nous supposons que l'étudiante ou l'étudiant les réalise. Nous pouvons les utiliser en référence dans les cours, les examens et les travaux pratiques.

Utilisation d'outils d'ingénierie

S.O.

Évaluation

Type d'évaluation	Pondération	Modalité	Description	Utilisation de l'IA
Examen intra	10	Individuel	Examen intra 1	Utilisation interdite
Examen intra	20	Individuel	Examen intra 2	Utilisation interdite
Examen final	40	Individuel	Examen final	Utilisation interdite
Devoir / exercice / travaux pratiques	15	En équipe	Travail pratique #1	À préciser pour chacune des activités évaluatives à la discrétion du titulaire du cours.
Devoir / exercice / travaux pratiques	15	En équipe	Travail pratique #2	À préciser pour chacune des activités évaluatives à la discrétion du titulaire du cours.

Informations additionnelles :

Évaluation	Pondération	Semaine
Examen intra 1 (2 heures)	10 %	5
Travail pratique #1	15 %	Groupe 01 : 04 novembre Groupe 02 : 06 novembre Groupes 03 et 05 : 03 novembre
Examen intra 2 (2 heures)	20 %	10
Travail pratique #2	15 %	Groupe 01 : 02 décembre Groupe 02 : 04 décembre Groupes 03 et 05 : 07 décembre
Examen final (3 heures)	40%	Période des examens finaux

À propos des travaux pratiques :

- Les travaux pratiques visent à mettre en application, dans le cadre d'un problème réaliste, les concepts et les méthodes de programmation enseignés pendant les cours théoriques. Il est attendu des étudiant.es de produire des travaux qui permettent de résoudre le problème de l'énoncé mais surtout de les réaliser selon les standards de qualité et les bonnes pratiques enseignés (respect des normes, décomposition en modules, etc.). La cohérence et l'optimalité des algorithmes implémentés constituent également des critères importants qui seront considérés. En tout état de cause, la note attribuée à l'exécution correcte du programme ne pourra excéder **40%** de la note globale d'un travail pratique.
- Chaque travail pratique se fera en équipes de 2 à 3 membres chacune. Une équipe ne doit remettre qu'un seul travail.
- L'utilisation de Git et de Github est fortement encouragée.
- Il est à noter que pour les travaux en équipe, chaque membre recevra une note inférieure ou égale à la note obtenue par l'équipe. Cette note individuelle est directement proportionnelle à l'implication de l'individu. La méthode d'évaluation de l'implication de chaque individu est laissée à la discrétion de l'enseignant.

À propos des examens intras :

- Les examens intra ont lieu en présence.
- Les examens intra ont lieu sur papier et/ou sur Enaquiz.
- Aucune documentation et aucun accès Internet ne sont permis.

À propos de l'examen final :

- L'examen final a lieu en présence.
- Aucune documentation et aucun accès Internet ne sont permis.
- L'énoncé de l'examen final sera distribué en format papier aux étudiantes et étudiants qui donneront leurs réponses sur Enaquiz et/ou sur papier.

Double seuil :

- Une note moyenne pondérée de 50 % est exigée pour l'ensemble des évaluations à caractère individuel. Ce seuil est une condition nécessaire à la réussite du cours mais ne la garantit pas. Seuls les examens intra et l'examen final sont utilisés dans le calcul du double seuil.

À propos de la langue utilisée

- Afin de respecter les exigences relatives à la langue française telles que stipulées à la Politique linguistique de l'École, le code et les commentaires de code doivent être principalement en français tel que c'est le cas pour tous les exemples et documents qui seront fournis aux étudiantes et étudiants.

Seuil de passage pour les éléments à caractère individuel

Note minimale : 50

Dates des examens intra

Groupe(s)	Date
1	30 septembre 2026
1	11 novembre 2026
2	13 novembre 2026
2	2 octobre 2026
3, 5	10 novembre 2026
3, 5	29 septembre 2026

Politique de retard des travaux

Conformément au [Règlement des études de premier cycle](#) (article 7.5.6) et au [Règlement des études de cycles supérieurs](#) (article 6.5.6), tout travail (devoir pratique, rapport de laboratoire, rapport de projet, etc.) remis en retard sans motif valable, c'est-à-dire autre que ceux mentionnés à l'article 7.5.5.1 dans le [Règlement des études de premier cycle](#) et l'article 6.5.2 dans le [Règlement des études de cycles supérieurs](#), se verra attribuer la note zéro, à moins que d'autres dispositions soient communiquées par écrit par la personne enseignante dans les consignes de chaque travail à remettre ou dans le plan de cours pour l'ensemble des travaux.

Absence à une évaluation

Afin de faire valider une absence à une évaluation en vue d'obtenir un examen de compensation, l'étudiante ou l'étudiant doit utiliser le [formulaire prévu à cet effet](#) dans son portail MonÉTS pour un examen final qui se déroule durant la période des examens finaux ou pour tout autre élément d'évaluation surveillé de 15% et plus durant la session. Si l'absence concerne un élément d'évaluation de moins de 15% durant la session, l'étudiant ou l'étudiante doit soumettre une demande par écrit à son enseignante ou enseignant. Toute demande de validation d'absence doit se faire dans les cinq (5) jours ouvrables suivant la tenue de l'évaluation, sauf dans les cas d'une absence pour participation à une activité prévue aux règlements des études où la demande doit être soumise dans les cinq (5) jours ouvrables avant le jour de départ de l'ÉTS pour se rendre à l'activité. Toute absence non justifiée par un motif majeur (voir articles 7.2.6.1 du [RÉPC](#) et 6.5.2 du [RÉCS](#)) entraînera l'attribution de la note zéro (0).

Infractions de nature académique

Les clauses du « Règlement sur les infractions de nature académique de l'ÉTS » s'appliquent dans ce cours ainsi que dans tous les cours du département. Les étudiantes et les étudiants doivent consulter le [Règlement sur les infractions de nature académique](#) (www.etsmtl.ca/a-propos/gouvernance/secretariat-general/cadre-reglementaire/reglement-sur-les-infractions-de-nature-academique) pour identifier les actes considérés comme étant des infractions de nature académique ainsi que prendre connaissance des sanctions prévues à cet effet. À l'ÉTS, le respect de la propriété intellectuelle est une valeur essentielle et tous les membres de la communauté étudiante sont invités à consulter la page [Citer, pas plagier !](#) (www.etsmtl.ca/Etudiants-actuels/Baccalaureat/Citer-pas-plagier).

Systèmes d'intelligence artificielle générative (SIAG)

L'utilisation des systèmes d'intelligence artificielle générative (SIAG) dans les activités d'évaluation constitue une infraction de nature académique au sens du Règlement sur les infractions de nature académique, sauf si elle est explicitement autorisée par la personne enseignante du cours ou la personne coordonnatrice dans le cas des stages.

Documentation obligatoire

Aucune référence obligatoire.

Ouvrages de références

Horstmann, Cay, *Big Java, 3rd Edition*, San Jose State Univ., John Wiley & sons, 2007, 1248 pages.
Horton's, Ivor, *Ivor Horton's beginning Java™ 2, JDK™5 Edition*, Indianapolis, Wiley Publishing, Inc., 2005, 1470 pages.

Adresse internet du site de cours et autres liens utiles

<https://ena.etsmtl.ca/course/view.php?id=537>
[Java version 8](#)

Autres informations

Les séances de cours et de travaux pratiques des cours-groupes dont le mode d'enseignement est hybride sont offertes entièrement à distance. L'étudiante ou l'étudiant inscrit à un tel cours-groupe n'a donc pas besoin de se déplacer à l'École durant la session, sauf lors des évaluations en présence identifiées à la section "Évaluation".

